Project 3

Due: Nov 8, 2026, at 11:59 pm (See BS for official due)

Please read the Common Projects Instructions first.

Problem 3.1: Parallel Matrix Multiplication Using Fox's and Strassen's Methods: Design, implement, and analyze Fox's and Strassen's algorithms for multiplying two large $\mathbf{N \times N}$ matrices **A** and **B** with random elements in (-1, 1]. Use any available parallel computer and interconnection network.

Tasks:

1. Briefly describe both algorithms.
2. Implement them on your parallel system. For Strassen's method, apply one or two levels of recursion and multiply the resulting blocks serially or with Fox's method; state how you map the work onto the P cores.
3. Test performance for:
   - Cores $\mathbf{P = 2^2, 2^4, 2^6}$ (4, 16, 64 cores)
   - Matrix sizes $\mathbf{N = 2^8, 2^{10}, 2^{12}}$ (256, 1024, 4096)
4. Collect timing data for all 9 (P, N) combinations.
5. Plot speedup curves (state your serial baseline), and verify correctness against a serial product.
6. Discuss performance trends and scalability.

Deliverables: Algorithm descriptions, source code, timing results, speedup plots, and a brief performance analysis.

References:

1. G. C. Fox, S. W. Otto, A. J. G. Hey, "Matrix algorithms on a hypercube I: Matrix multiplication," Parallel Computing 4(1), 17–31 (1987). https://doi.org/10.1016/0167-8191(87)90060-3

2. V. Strassen, "Gaussian elimination is not optimal," Numerische Mathematik 13, 354–356 (1969). https://doi.org/10.1007/BF02165411

Problem 3.2: Parallel Matrix Multiplication Using Cannon's and Fox's Methods: Design, implement, and analyze these two algorithms for multiplying two large $\mathbf{N \times N}$ matrices **A** and **B** with random elements in (-1, 1]. Use any available parallel computer and interconnection network.

Tasks:

1. Briefly describe both algorithms.
2. Implement them on your parallel system.
3. Test performance for:
   - Cores $\mathbf{P = 2^2, 2^4, 2^6}$ (4, 16, 64 cores)
   - Matrix sizes $\mathbf{N = 2^8, 2^{10}, 2^{12}}$ (256, 1024, 4096)
4. Collect timing data for all 9 (P, N) combinations.
5. Plot speedup curves and compare them.
6. Discuss performance differences, trends, and scalability.

Deliverables: Algorithm descriptions, source code, timing results, speedup plots, and a performance analysis, especially a comparison of the two in terms of scalability.

Note: You may reuse your Fox code between Problems 3.1 and 3.2; you are encouraged to compare all three methods (Fox, Cannon, and Strassen) within your chosen problem.

References:

1. L. E. Cannon, A Cellular Computer to Implement the Kalman Filter Algorithm, Ph.D. thesis, Montana State University (1969).

2. G. C. Fox, S. W. Otto, A. J. G. Hey, "Matrix algorithms on a hypercube I: Matrix multiplication," Parallel Computing 4(1), 17–31 (1987). https://doi.org/10.1016/0167-8191(87)90060-3

Problem 3.3: This problem focuses on optimizing communication in a two-level Strassen matrix multiplication (MM) for matrices $A_{N\times N}$ and $B_{N\times N}$, where N is a large power of 2, e.g., $N = 2^{10} = 1024$.

One-Level Strassen MM: In Strassen's algorithm, matrices $\boldsymbol{A_{N\times N}}$ and $\boldsymbol{B_{N\times N}}$ are decomposed into $\mathbf{2\times 2}$ submatrices. This decomposition enables Strassen's algorithm to perform 7 submatrix multiplications **instead of the 8 required by naïve matrix multiplication, reducing computational complexity.**

Two-Level Strassen MM: For a two-level Strassen MM, each submatrix is further decomposed recursively into a grid of sub-sub-matrices (ss-matrices) which results in 16 ss-matrices per input matrix (32 total for A and B). The two-level Strassen algorithm requires 49 ss-matrix multiplications instead of the 64 needed in naïve MM.

$$\boldsymbol{A} = \begin{bmatrix} \boldsymbol{A_{11}} & \boldsymbol{A_{12}} \\ \boldsymbol{A_{21}} & \boldsymbol{A_{22}} \end{bmatrix} = \begin{bmatrix} \boldsymbol{a_{11}} & \boldsymbol{a_{12}} & \boldsymbol{a_{13}} & \boldsymbol{a_{14}} \\ \boldsymbol{a_{21}} & \boldsymbol{a_{22}} & \boldsymbol{a_{23}} & \boldsymbol{a_{24}} \\ \boldsymbol{a_{31}} & \boldsymbol{a_{32}} & \boldsymbol{a_{33}} & \boldsymbol{a_{34}} \\ \boldsymbol{a_{41}} & \boldsymbol{a_{42}} & \boldsymbol{a_{43}} & \boldsymbol{a_{44}} \end{bmatrix}$$

$$\boldsymbol{B} = \begin{bmatrix} \boldsymbol{B_{11}} & \boldsymbol{B_{12}} \\ \boldsymbol{B_{21}} & \boldsymbol{B_{22}} \end{bmatrix} = \begin{bmatrix} \boldsymbol{b_{11}} & \boldsymbol{b_{12}} & \boldsymbol{b_{13}} & \boldsymbol{b_{14}} \\ \boldsymbol{b_{21}} & \boldsymbol{b_{22}} & \boldsymbol{b_{23}} & \boldsymbol{b_{24}} \\ \boldsymbol{b_{31}} & \boldsymbol{b_{32}} & \boldsymbol{b_{33}} & \boldsymbol{b_{34}} \\ \boldsymbol{b_{41}} & \boldsymbol{b_{42}} & \boldsymbol{b_{43}} & \boldsymbol{b_{44}} \end{bmatrix}$$

Task: You are given 49 processing cores, each assigned one of the 49 ss-matrix multiplications. Initially, the 32 ss-matrices are distributed among the cores, at most two per core. However, each of the 49 multiplications may require up to 8 ss-matrices (as described in Lecture Notes, Topic 6, p. 59). Since at most two ss-matrices are pre-allocated per core, the remaining ss-matrices must be fetched, necessitating an efficient communication scheme to minimize data transfer overhead.

Objective: Design a quasi-optimal communication scheme to complete the two-level Strassen MM, ensuring that each core has the necessary ss-matrices to perform its assigned multiplication while minimizing communication costs. Implement your algorithm on a suitable computing platform.

References:

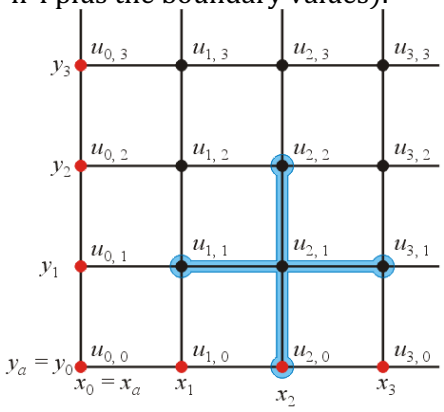
1. A. Fawzi et al.: *Discovering faster matrix multiplication algorithms with reinforcement learning*. Nature 610, 47–53 (2022).
2. Chou, Li, Deng, Wang: *Parallelizing Strassen's method for matrix multiplication on distributed-memory MIMD architectures*
3. OpenAI: An upper bound of 9/4 for the matrix multiplication exponent, preprint, October 2, 2026.

Notes: (1) This problem, along with Problems 4.3 and 5.3, lacks the usual generality and flexibility of mathematical problems. (2) As an incentive for selecting this problem, 4.3, or 5.3, I will boost your letter grade by one notch, e.g., from B+ to A-, based on the normalized numerical grades. You get one, and only one, such lift even if you select more than one from 3.3, 4.3, 5.3. (3) For 5.3, some students will be involuntarily assigned and will automatically receive the one-notch letter grade boost.

Problem 3.4: 2D Poisson equation with boundary conditions,

$$\begin{cases} u_{xx} + u_{yy} = f(x,y) \\ u(\partial\Omega) = U \end{cases}$$

can be discretized, with the standard five-point finite-difference scheme, as a sparse system of equations (with this matrix, b = −h²f plus the boundary values):



Solve the following by two methods:

$$A\boldsymbol{u} = \begin{bmatrix} 4 & -1 & 0 & 0 & -\boldsymbol{I}_{4\times4} & \boldsymbol{0}_{4\times4} & \boldsymbol{0}_{4\times4} \\ -1 & 4 & -1 & 0 & & & \\ 0 & -1 & 4 & -1 & & & \\ 0 & 0 & -1 & 4 & & & \\ -\boldsymbol{I}_{4\times4} & & & & 4 & -1 & 0 & 0 & -\boldsymbol{I}_{4\times4} & \boldsymbol{0}_{4\times4} \\ & & & & -1 & 4 & -1 & 0 & & \\ & & & & 0 & -1 & 4 & -1 & & \\ & & & & 0 & 0 & -1 & 4 & & \\ \boldsymbol{0}_{4\times4} & & & & -\boldsymbol{I}_{4\times4} & & & & 4 & -1 & 0 & 0 & -\boldsymbol{I}_{4\times4} \\ & & & & & & & & -1 & 4 & -1 & 0 & \\ & & & & & & & & 0 & -1 & 4 & -1 & \\ & & & & & & & & 0 & 0 & -1 & 4 & \\ \boldsymbol{0}_{4\times4} & & & & \boldsymbol{0}_{4\times4} & & & & -\boldsymbol{I}_{4\times4} & & & & \begin{matrix} 4 & -1 & 0 & 0 \\ -1 & 4 & -1 & 0 \\ 0 & -1 & 4 & -1 \\ 0 & 0 & -1 & 4 \end{matrix} \end{bmatrix} \begin{bmatrix} u_{00} \\ u_{10} \\ u_{20} \\ u_{30} \\ u_{01} \\ u_{11} \\ u_{21} \\ u_{31} \\ u_{02} \\ u_{12} \\ u_{22} \\ u_{32} \\ u_{03} \\ u_{13} \\ u_{23} \\ u_{33} \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \\ 0 \end{bmatrix} = \boldsymbol{b}$$

(a) Jacobi method and (b) Gauss-Seidel method. For both methods, you may select the initial $[\boldsymbol{u}^{(0)}] = [\boldsymbol{0}]$ (bold symbols denote vectors). Iterate a reasonable number of steps (K~10) on this 16-unknown example and, for each step of each method, report:

$$\epsilon(k) = \left\| A\boldsymbol{u}^{(k)} - \boldsymbol{b} \right\|_{\infty}, \qquad \forall k \le K$$

The example above discretizes the domain Ω in a 4 × 4 grid of interior points (16 unknowns). For the parallel runs, use a mesh no smaller than 128 × 128 and you should use $P = 1, 2, 4, 16$ cores. Time a fixed number of iterations (e.g., 1,000) or iterate to a tolerance, and report the time per iteration; for Gauss–Seidel, use red–black ordering so that the updates can be parallelized. Implement the algorithm on a parallel system (e.g., SeaWulf) **and** complete the following:

(1) Design and implement the parallel algorithm.
(2) Collect the performance results and analyze them.
(3) Plot the speedup curves.

Problem 3.5 (contributed by A.W.): **Pricing of 2-Asset American Options by Projected SOR.**

The value $V(S_1, S_2, t)$ of an American option on two uncorrelated assets satisfies the Black–Scholes PDE

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma_1^2 S_1^2 \frac{\partial^2 V}{\partial S_1^2} + \frac{1}{2}\sigma_2^2 S_2^2 \frac{\partial^2 V}{\partial S_2^2} + rS_1 \frac{\partial V}{\partial S_1} + rS_2 \frac{\partial V}{\partial S_2} - rV = 0$$

Discretized with the five-point stencil of Problem 3.4, it yields a sparse system of the same structure. Unlike 3.4, the solver marches backward in time and solves one such system per time step. Solve each system by projected SOR: after every update, set

$$V_{ij} \leftarrow \max(V_{ij}, \Phi_{ij})$$

where Φ is the payoff. This step enforces the holder's right to exercise early.

Tasks. (1) Parallelize each solve with 2D domain decomposition and red–black ordering, as in 3.4. (2) Run meshes of 128×128 and larger on P = 1, 2, 4, 16 (and 64) cores, and plot the speedup curves. (3) Verify correctness against a serial run and against the closed-form European price, obtained by skipping the max() step.

In short, this is Problem 3.4 with many sequential solves.